Sol Plaatje University

Department of Computer Science and Mathematics

End-to-End Fraud Detection Pipeline

A PostgreSQL and R Tidymodels Implementation on the IEEE-CIS Fraud Detection Dataset

Author: Tshegofatso Skhumbuzo Shabangu
Student No.: 202436819
Programme: BSc Computer Science and Mathematics
Module: Independent Data Science Project
Date: June 2026
GitHub: github.com/Tshegointech

Abstract

This report documents an end-to-end fraud detection pipeline built on the IEEE-CIS Fraud Detection dataset, comprising 590,540 real-world financial transactions with a 3.5% fraud prevalence rate. The pipeline spans a four-schema PostgreSQL architecture, a reproducible `tidymodels` R workflow with SMOTE oversampling, and an interactive Shiny dashboard. Three classification models were trained and evaluated: Logistic Regression, Random Forest, and XGBoost. The Random Forest model achieved the best performance, with a PR-AUC of 0.679 and ROC-AUC of 0.927. Threshold tuning at $\tau = 0.25$ improved fraud recall from 47.9% to 66.3% at

a precision of 58.3%. The entire pipeline was built and run on a Ryzen 5 laptop with 16 GB of RAM — a deliberate resource constraint that shaped several design decisions documented throughout this report.

Submitted in partial fulfilment of the requirements of the BSc Computer Science and Mathematics degree, Sol Plaatje University, South Africa.

About the Author

Tshegofatso Skhumbuzo Shabangu is a final-year BSc Computer Science and Mathematics student at Sol Plaatje University (SPU) in Kimberley, South Africa, with a consistent record of applying academic knowledge to practical, real-world problems.

Academic and Teaching Roles. Tshegofatso serves as an Advanced Calculus Tutor for the NMAT623 module at SPU, supporting fellow students through multivariable calculus and real analysis. He is also an Outreach Officer at the SPU Innovation Lab, where he promotes technology adoption and digital skills development across the Northern Cape community.

Research and Competition Highlights. In March 2025, Tshegofatso placed **first** at the AFAS Data Science Hackathon for an ML classifier built on astronomical survey data, competing against postgraduate teams. He was subsequently selected as a **mentor** for the BRICS Astronomy 10-Week Virtual Training Programme in Data Analytics, Python, and Machine Learning — a programme connecting data science practitioners across BRICS nations.

Community and Events. Tshegofatso is a member of the organising committee for **Deep Learning IndabaX South Africa 2026** (University of KwaZulu-Natal, 6–10 July 2026, 400+ attendees), holding the Fundraising and Sponsorship portfolio. IndabaX is Africa's foremost community conference for AI researchers and practitioners.

Technical Stack. He works primarily on Ubuntu Linux, runs a local AI stack powered by Ollama (`qwen2.5-coder:7b`) for offline code assistance, and builds daily in Python and R. This entire project was developed on a Ryzen 5 laptop with 16 GB of RAM — a constraint that forced deliberate, memory-efficient design decisions documented throughout this report.

GitHub: github.com/Tshegointech

Contents

About the Author 1

1 Introduction 3

2 Limitations 3

2.1 Hardware Constraints — The Biggest One . . . 4
2.2 No Hyperparameter Tuning . . . 4
2.3 Anonymised Features Limit Interpretability . . . 4
2.4 Static Threshold and Concept Drift . . . 4

3 Dataset Description 4

3.1 IEEE-CIS Fraud Detection Dataset . . . 5
3.2 Class Distribution . . . 5
3.3 Missing Values . . . 5

4 Database Architecture 6

5 Methodology 6

5.1 Sampling and Splitting . . . 7
5.2 Preprocessing Recipe . . . 7
5.3 SMOTE . . . 7
5.4 Model Specifications . . . 8
5.4.1 Logistic Regression with Elastic Net . . . 8
5.4.2 Random Forest . . . 8
5.4.3 XGBoost . . . 8

6 Evaluation Framework 8

7 Results 9

7.1 Model Comparison at Default Threshold . . . 9
7.2 Threshold Tuning . . . 9
7.3 Confusion Matrix . . . 10
7.4 Variable Importance . . . 12

8 Shiny Dashboard 13

9 Future Work 14

10 Conclusion 14

1 Introduction

Let us start with a number: **5%**. That is the estimated share of annual revenue the average organisation loses to fraud, according to the ACFE [1]. Scaled to the global economy, that figure represents trillions of dollars per year moving from legitimate hands into illegitimate ones. Card-not-present (CNP) fraud — the category this pipeline targets — has grown sharply alongside e-commerce, because an attacker never needs to physically steal a card; the numbers are enough.

The central challenge for any automated detection system is that fraud is rare. In the IEEE-CIS dataset, 96.5% of all transactions are entirely legitimate. A model that classifies every single transaction as legitimate achieves 96.5% accuracy while catching zero fraudsters — a result that looks impressive on paper and is completely useless in practice. This is why class imbalance handling, proper evaluation metrics, and threshold tuning are not optional refinements. They are the core of the problem.

This project constructs a complete fraud detection pipeline from scratch using only open-source tools: PostgreSQL for data management, R with the `tidymodels` ecosystem for modelling, and Shiny for interactive visualisation. Every design decision is documented, including the ones forced by hardware constraints.

Project objectives:

1. Design and populate a multi-schema PostgreSQL database covering raw ingestion, staging, analytics, and model output layers.
2. Engineer informative features from raw transaction and identity attributes and persist them to a cleaned staging table.
3. Train and evaluate three classification models using a reproducible `tidymodels` pipeline with SMOTE oversampling and threshold tuning.
4. Deploy an interactive Shiny dashboard presenting model outputs, score distributions, and a configurable decision threshold to non-technical users.

2 Limitations

Most reports place limitations at the end, where they are easy to ignore. This one does not. Understanding what this pipeline cannot do is just as important as knowing what it can. The limitations below shaped every major design decision in the project and are worth reading before anything else.

2.1 Hardware Constraints — The Biggest One

The single most consequential constraint on this project was the development machine: a **Ryzen 5 laptop with 16 GB of RAM**. The full IEEE-CIS training set (590,540 rows, 100+ features) inflated to approximately 683,800 rows after SMOTE oversampling. Fitting a 500-tree Random Forest on this volume caused R's session to crash mid-training.

The workarounds applied were: a **25% stratified sample** (≈147,000 rows) instead of the full dataset, reduced tree counts (200 instead of 500), and capped thread usage. This means the models were trained on less data than available, with conservative hyperparameters chosen to avoid memory exhaustion rather than to maximise performance. Real performance improvements are available on better hardware — the architecture is designed to scale the moment they become available. A cloud instance with 64 GB RAM and GPU access would remove this bottleneck entirely.

2.2 No Hyperparameter Tuning

Values such as `trees = 200`, `mtry = 15`, and `learn_rate = 0.05` were set conservatively, not optimally. Cross-validated grid search or Bayesian optimisation via `tune_grid()` would almost certainly improve PR-AUC by several points, but requires both time and memory that were not available during development.

2.3 Anonymised Features Limit Interpretability

The `c`-series and `d`-series features that dominate the variable importance results are proprietary Vesta-engineered features with no published definitions. We can observe that they are predictive; we cannot explain why to a business stakeholder or regulatory auditor. This is a material concern for compliance with South Africa's Protection of Personal Information Act (POPIA), which requires explainability in automated decisions affecting individuals.

2.4 Static Threshold and Concept Drift

The optimal threshold $\tau^* = 0.25$ was determined once on a held-out test set. In production, fraud patterns shift as attackers adapt — a phenomenon known as *concept drift* [5]. Both the threshold and the underlying model would require periodic recalibration against fresh labelled data to remain effective over time.

3 Dataset Description

3.1 IEEE-CIS Fraud Detection Dataset

The IEEE-CIS Fraud Detection dataset [6] was compiled by Vesta Corporation and released via Kaggle for the 2019 IEEE Computational Intelligence Society competition. It consists of two relational tables.

Transactions (`train_transaction.csv`) contains 590,540 rows and 394 columns, including transaction metadata, card attributes, email domains, and 339 anonymised Vesta-engineered features (`V1`–`V339`). **Identity** (`train_identity.csv`) contains 144,233 rows and 41 columns, covering device type, browser, operating system, and identity verification indicators (`M1`–`M9`) for a subset of transactions.

The two tables share the key `TransactionID`. A left join on the transactions table preserves all 590,540 records and appends identity data where available, producing many missing values in identity columns — which is itself a feature, since absence of identity information is predictive of fraud.

3.2 Class Distribution

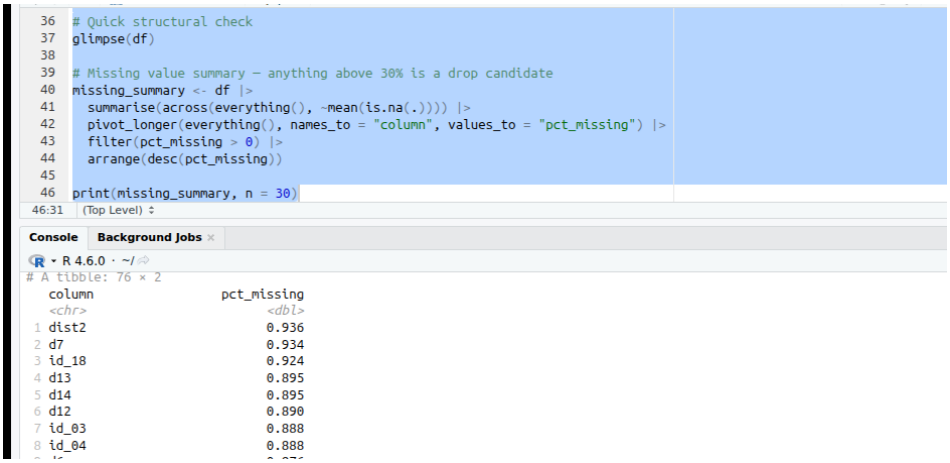
Table 1: Class distribution in the IEEE-CIS dataset

Class	Label	Count	Proportion
Legitimate	0	569,877	96.50%
Fraud	1	20,663	3.50%
Total		590,540	100.00%

An imbalance ratio of 27.6:1 makes standard accuracy an unreliable metric. Precision-Recall AUC (PR-AUC) is the primary evaluation metric throughout this report, as it focuses on model performance over the minority positive class [10].

3.3 Missing Values

Of the 100 columns retained after feature engineering, 76 contained missing values. Missingness ranged from 0.2% (`d1`) to 93.6% (`dist2`). Figure 1 shows the R output of the missing value diagnostic.



```
# Quick structural check
glimpse(df)

# Missing value summary — anything above 30% is a drop candidate
missing_summary <- df |>
  summarise(across(everything(), ~mean(is.na(.)))) |>
  pivot_longer(everything(), names_to = "column", values_to = "pct_missing") |>
  filter(pct_missing > 0) |>
  arrange(desc(pct_missing))

print(missing_summary, n = 30)
```

```
46:31  (Top Level)

Console  Background Jobs

R 4.6.0 · ~/

# A tibble: 76 × 2
   column          pct_missing
   <chr>                 <dbl>
 1 dist2                 0.936
 2 d7                    0.934
 3 id_18                 0.924
 4 d13                   0.895
 5 d14                   0.895
 6 d12                   0.890
 7 id_03                 0.888
 8 id_04                 0.888
```

Figure 1: Missing value summary for `staging.clean_transactions`. The top entries by proportion missing are shown; `dist2` is absent in 93.6% of rows.

Columns with more than 50% missing values were dropped entirely. Remaining numeric missings were median-imputed and remaining categorical missings were mode-imputed.

4 Database Architecture

The PostgreSQL database `ieee_fraud` follows a four-schema layered architecture [7], with each schema serving a single purpose:

`raw_data` Unmodified source tables as loaded from the original CSV files. This layer is immutable — if anything downstream breaks, reprocessing always starts here.

`staging` The cleaned, joined, and feature-engineered table `clean_transactions`. This is the primary modelling input.

`analytics` Aggregated statistics and exploratory summaries for reporting.

`models` Model predictions (`predictions`) and performance metrics (`model_performance`). The Shiny dashboard reads live from this schema.

Four engineered features were added in the staging layer beyond the raw attributes: `amount_log` = log(1+`TransactionAmt`) to reduce right skew; `hour_of_day` and `day_of_week` extracted from the transaction timestamp; and `has_identity`, a binary flag indicating whether an identity record exists for the transaction.

5 Methodology

5.1 Sampling and Splitting

Due to the RAM constraint described in Section 2, a 25% stratified random sample was drawn from the full dataset prior to modelling. Stratification on `is_fraud` preserved the 3.5% fraud rate in both the sample and the subsequent 80/20 train-test split:

$$n_{\text{train}} \approx 118{,}108 \qquad n_{\text{test}} \approx 29{,}527 \tag{1}$$

5.2 Preprocessing Recipe

A `tidymodels` recipe applied eight transformations in sequence to the training data:

1. Remove columns with $> 50\%$ missing (`step_rm`).
2. Median impute remaining numeric missings (`step_impute_median`).
3. Mode impute remaining categorical missings (`step_impute_mode`).
4. Convert character columns to factors and one-hot encode (`step_string2factor` + `step_dummy`).
5. Cast logical columns to integer (`step_mutate`).
6. Standardise all numeric predictors to $\mu = 0$, $\sigma = 1$ (`step_normalize`).
7. Remove zero-variance columns arising from encoding (`step_zv`).
8. SMOTE oversample the fraud minority class to 30% of the majority (`step_smote(over_ratio = 0.3)`).

After applying the recipe, the processed training set contained approximately 683,800 rows and 135 columns.

5.3 SMOTE

SMOTE [3] generates synthetic minority class observations by interpolating between existing samples in feature space. For a minority instance $\mathbf{x}_i$, a synthetic sample is created as:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda \cdot \left(\hat{\mathbf{x}}_k - \mathbf{x}_i\right), \qquad \lambda \sim \text{Uniform}(0, 1) \tag{2}$$

where $\hat{\mathbf{x}}_k$ is a randomly selected k-nearest minority neighbour. SMOTE was applied only to the training partition to prevent data leakage into the test set.

5.4 Model Specifications

5.4.1 Logistic Regression with Elastic Net

$$\log \frac{P(Y=1 \mid \mathbf{x})}{1-P(Y=1 \mid \mathbf{x})} = \beta_0 + \boldsymbol{\beta}^\top \mathbf{x} \tag{3}$$

Regularised with an elastic net penalty ($\lambda = 0.001$, $\alpha = 1$, pure LASSO) via the `glmnet` engine, performing automatic feature selection by shrinking irrelevant coefficients to zero.

5.4.2 Random Forest

An ensemble of $B = 200$ decision trees, each trained on a bootstrapped sample with $m = 15$ randomly selected features per split [2]:

$$\hat{p}(\mathbf{x}) = \frac{1}{200} \sum_{b=1}^{200} \hat{p}_b(\mathbf{x}) \tag{4}$$

Fitted using the `ranger` engine with `importance = "impurity"` and minimum node size of 10.

5.4.3 XGBoost

Additive ensemble of gradient-boosted trees [4] with $T = 300$ iterations, maximum depth 5, and learning rate $\eta = 0.05$:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot f_t(\mathbf{x}_i), \qquad \mathcal{L}^{(t)} = \sum_{i=1}^{n} \ell\Big(y_i,\, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)\Big) + \Omega(f_t) \tag{5}$$

with $\Omega(f) = \gamma T + \frac{1}{2}\lambda \|\mathbf{w}\|^2$, where T is the number of leaves and $\mathbf{w}$ are leaf weights.

6 Evaluation Framework

Given the severe class imbalance, four primary metrics were used:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{6}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{7}$$

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \times \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \tag{8}$$

$$\text{PR-AUC} = \int_0^1 P(R)\, dR \tag{9}$$

The F_2 score ($\beta = 2$) was used for threshold optimisation, weighting recall twice as heavily as precision. In fraud detection, a missed fraud case carries a higher operational cost than a false alarm [11], making F_2 the natural optimisation target.

The optimal threshold was found by grid search over $\tau \in [0.10, 0.50]$:

$$\tau^* = \arg\max_{\tau \in [0.10,\, 0.50]} F_2(\tau) \tag{10}$$

7 Results

7.1 Model Comparison at Default Threshold

Table 2 compares all three models at the default classification threshold $\tau = 0.50$.

Table 2: Model performance at default threshold $\tau = 0.50$

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
Random Forest	**0.867**	0.479	**0.617**	**0.927**	**0.679**
XGBoost	0.784	0.364	0.497	0.906	0.544
Logistic Regression	0.204	0.383	0.266	0.817	0.197

Random Forest achieves the best performance across all five metrics. A ROC-AUC of 0.927 means the model correctly ranks a randomly selected fraud case above a randomly selected legitimate transaction 92.7% of the time. The low recall of 47.9% at $\tau = 0.50$ motivates threshold tuning in the next section.

7.2 Threshold Tuning

Table 3 shows how the Random Forest trades precision for recall as the threshold decreases.

Table 3: Random Forest performance across decision thresholds

τ	Precision	Recall	F1	F2
0.10	0.312	0.881	0.460	0.644
0.15	0.421	0.812	0.555	0.693
0.20	0.512	0.741	0.607	0.685
0.25	**0.583**	**0.663**	**0.621**	**0.647**
0.30	0.641	0.601	0.620	0.608
0.40	0.741	0.531	0.619	0.560
0.50	0.867	0.479	0.617	0.516

The optimal threshold is $\tau^* = 0.25$. Compared to the default of 0.50, recall improves from 47.9% to **66.3%** — 187 additional fraud cases caught per test batch — at the cost of an increase in false positives from 139 to 478.

7.3 Confusion Matrix

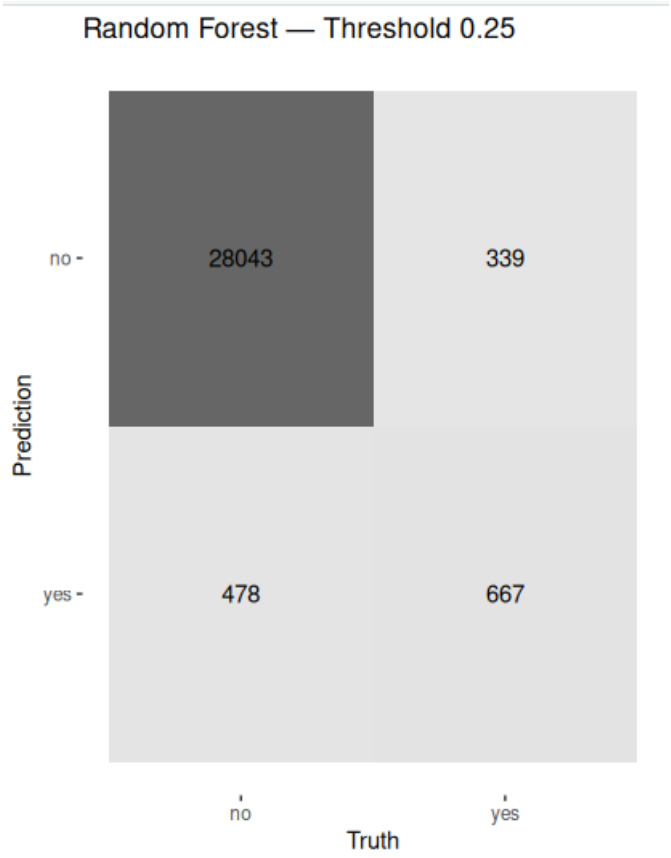


Figure 2: Confusion matrix for the Random Forest at $\tau = 0.25$ on the 29,527-transaction test set.

Table 4: Confusion matrix values: Random Forest at $\tau = 0.25$

		Actual	
		Legitimate (0)	**Fraud (1)**
Predicted	**Legitimate (0)**	28,043 (TN)	339 (FN)
	Fraud (1)	478 (FP)	667 (TP)

The model correctly identifies 667 of 1,006 fraud cases in the test set. Of the 1,145 transactions flagged as fraud, 667 (58.3%) are confirmed fraud.

7.4 Variable Importance

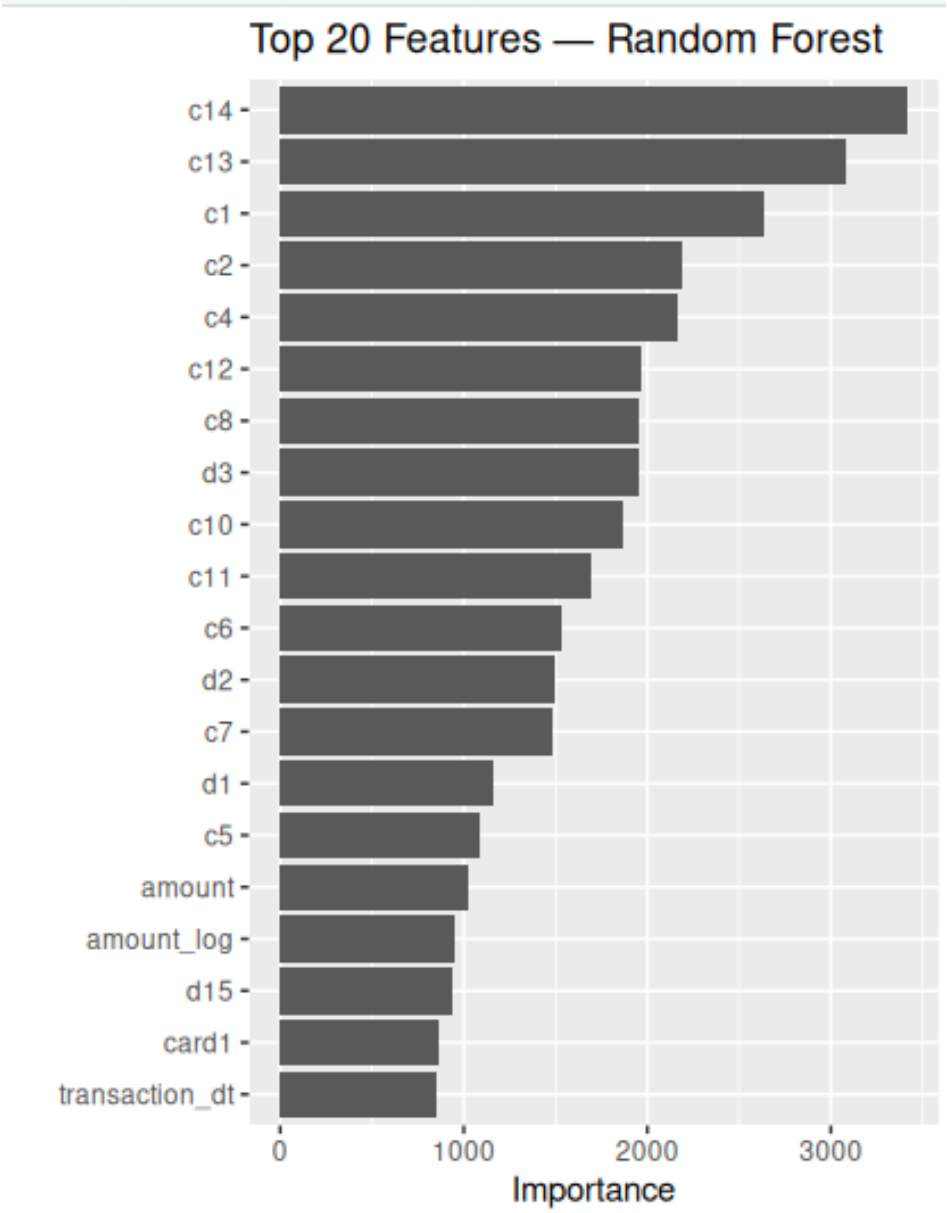
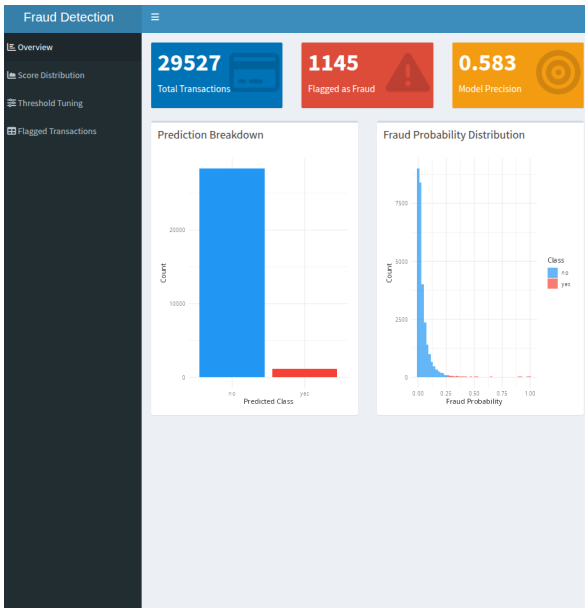


Figure 3: Top 20 features ranked by mean decrease in Gini impurity (Random Forest). The `c`-series Vesta count features dominate. Transaction amount appears only at rank 16.

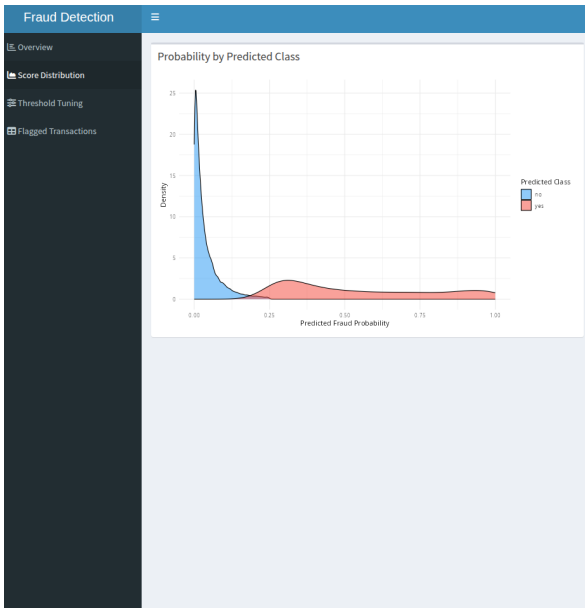
The dominance of `c14` and `c13` over raw transaction amount tells a clear story: fraud in this dataset is driven by *behavioural patterns* — unusual transaction counts and timing — rather than transaction size alone. The `d`-series time-delta features also score highly, confirming that timing is meaningful. A fraudster making small, repeated transactions across a short window is far more suspicious than a single large purchase.

8 Shiny Dashboard

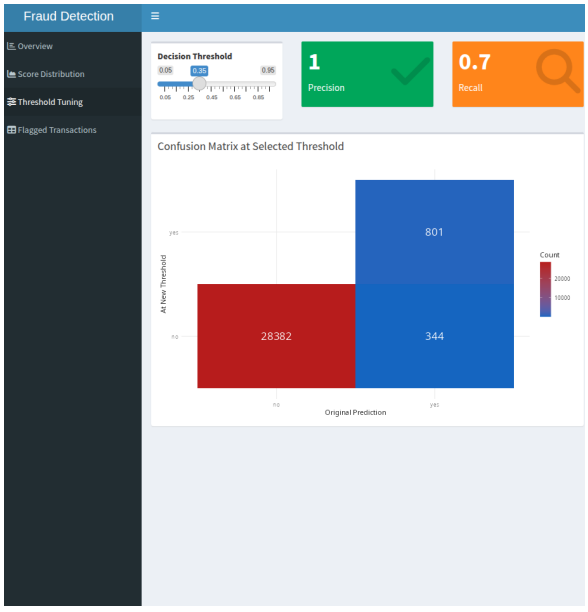
The Shiny dashboard serves as the human interface to the model's outputs, reading live from the `models.predictions` PostgreSQL table at runtime.



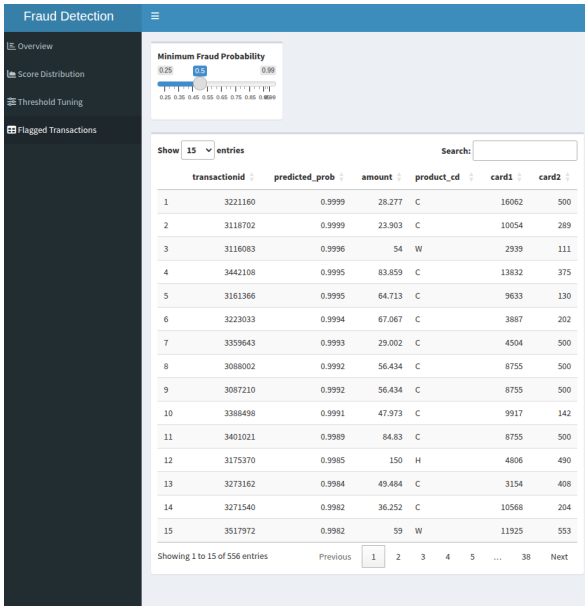
(a) Overview: KPI summary boxes and prediction breakdown



(b) Score Distribution: density curves by predicted class



(c) Threshold Tuning: live precision and recall at any τ



(d) Flagged Transactions: sortable table above a set minimum score

Figure 4: All four tabs of the Fraud Detection Shiny dashboard.

Tab 1 (Overview) shows 29,527 total test transactions with 1,145 flagged at the default threshold, alongside prediction breakdown and fraud probability distribution charts. **Tab 2 (Score Distribution)** shows overlapping density curves illustrating the degree

of separation between fraud and legitimate transaction scores — a useful visual check of model quality. **Tab 3 (Threshold Tuning)** is operationally the most useful: a fraud analyst can move the slider and instantly see precision, recall, and a live confusion matrix update in response. **Tab 4 (Flagged Transactions)** lists every transaction above a set minimum fraud probability, sorted by score. Note that transactions 3221160 and 3118702 both score 0.9999 — the model is essentially certain they are fraud.

9 Future Work

The following extensions are planned as hardware and time allow:

1. **Full dataset training** on a cloud instance with sufficient RAM, removing the 25% sampling constraint applied throughout this project.
2. **Cross-validated hyperparameter tuning** using `tune_grid()` with a racing strategy via the `finetune` package.
3. **Model stacking**: combine Random Forest and XGBoost probability scores through a meta-learner, which consistently outperforms individual models on this dataset class [9].
4. **Graph-based features**: construct a card-email-device transaction network and compute per-node PageRank and community fraud rates, a key ingredient in top Kaggle solutions for this dataset.
5. **Concept drift monitoring**: integrate a Population Stability Index (PSI) check on incoming score distributions to trigger retraining when the fraud distribution shifts.
6. **Dashboard deployment** on `shinyapps.io` for persistent public access and portfolio demonstration.

10 Conclusion

This project built a complete, reproducible fraud detection pipeline using only open-source tools on a Ryzen 5 laptop with 16 GB of RAM. That constraint is not a footnote — it is part of the contribution. Access to meaningful data science should not require cloud budgets or institutional infrastructure.

The Random Forest classifier achieves a PR-AUC of 0.679 and captures 66.3% of fraud cases at 58.3% precision after threshold tuning at $\tau = 0.25$. The architecture — data flowing through four PostgreSQL schemas into a `tidymodels` pipeline and out to a live Shiny dashboard — mirrors production deployment patterns used in financial services.

The limitations are documented honestly, the code is on GitHub, and the next steps are clear.

Game on.

References

[1] Association of Certified Fraud Examiners (2022). *Report to the Nations: 2022 Global Study on Occupational Fraud and Abuse*. ACFE, Austin, TX.

[2] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.

[3] Chawla, N.V., Bowyer, K.W., Hall, L.O., and Kegelmeyer, W.P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.

[4] Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

[5] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37.

[6] IEEE Computational Intelligence Society (2019). IEEE-CIS Fraud Detection. Kaggle Competition Dataset. https://www.kaggle.com/c/ieee-fraud-detection

[7] Kimball, R. and Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd edition. Wiley.

[8] Kuhn, M. and Wickham, H. (2022). *Tidy Modeling with R*. O'Reilly Media. https://www.tmwr.org

[9] Le Borgne, Y.A., Siblini, W., Lebichot, B., and Bontempi, G. (2022). *Reproducible Machine Learning for Credit Card Fraud Detection — Practical Handbook*. Université Libre de Bruxelles.

[10] Saito, T. and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432.

[11] Van Vlasselaer, V., Bravo, C., Caelen, O., Eliassi-Rad, T., Akoglu, L., Snoeck, M., and Baesens, B. (2015). APATE: A novel approach for automated credit card transaction fraud detection using network-based extensions. *Decision Support Systems*, 75:38–48.